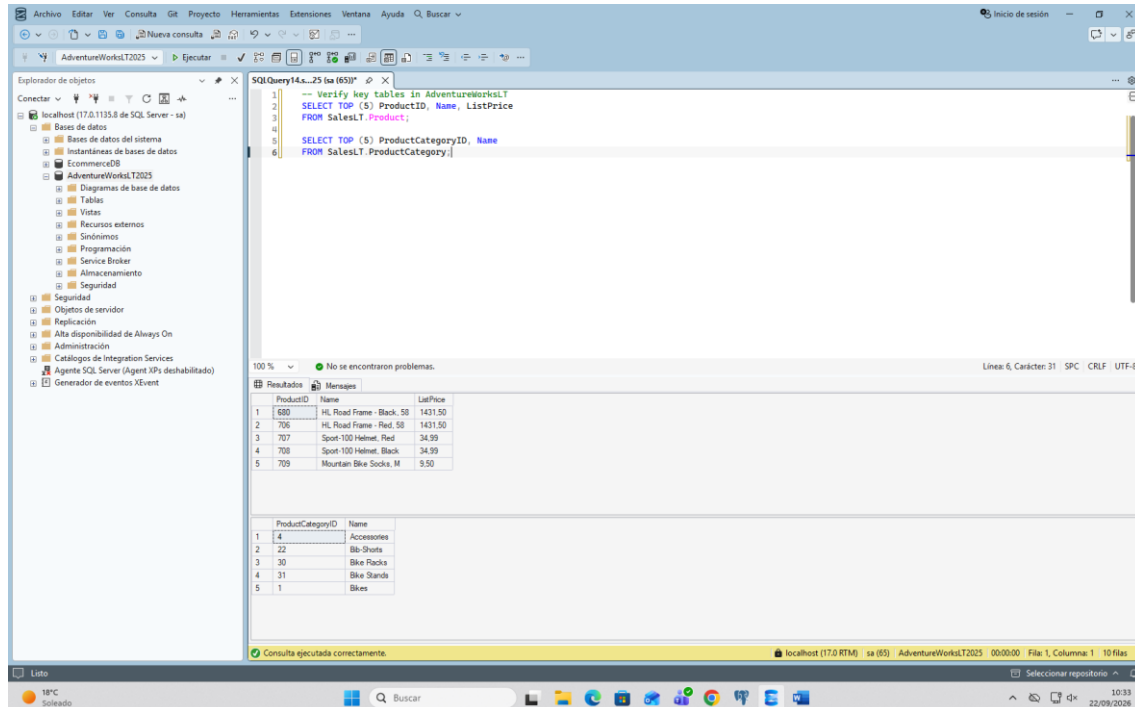


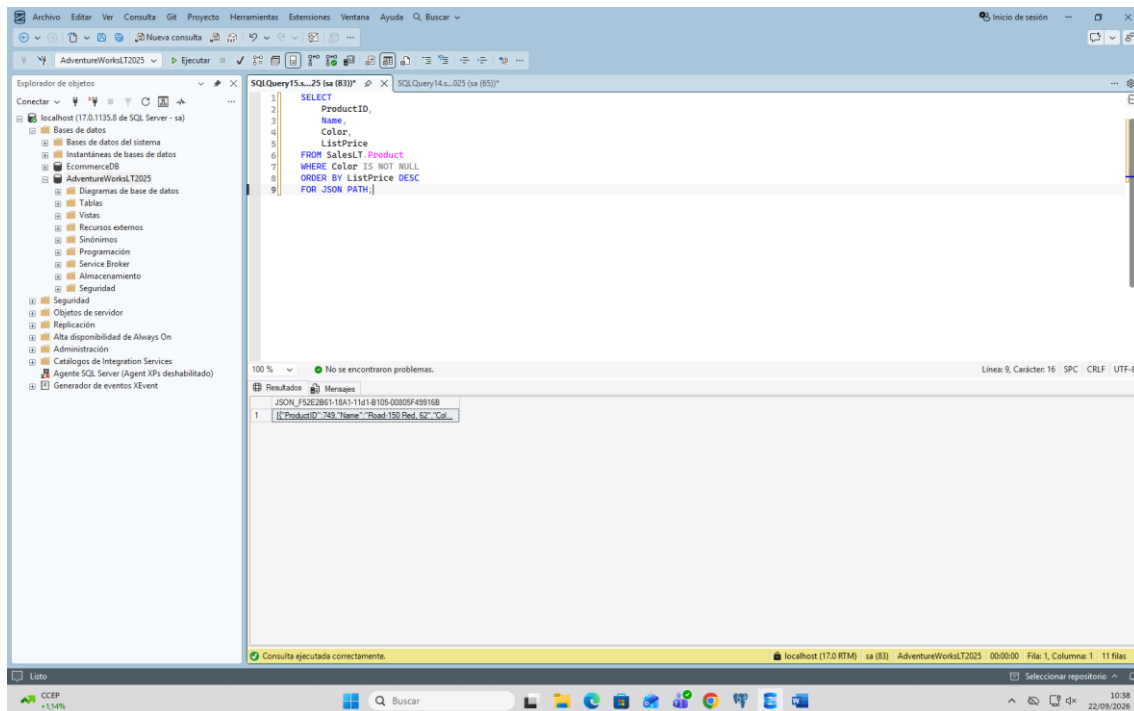
<https://github.com/alexmobe365/DP-800-Lab-2>

Conectarse a AdventureWorksLT



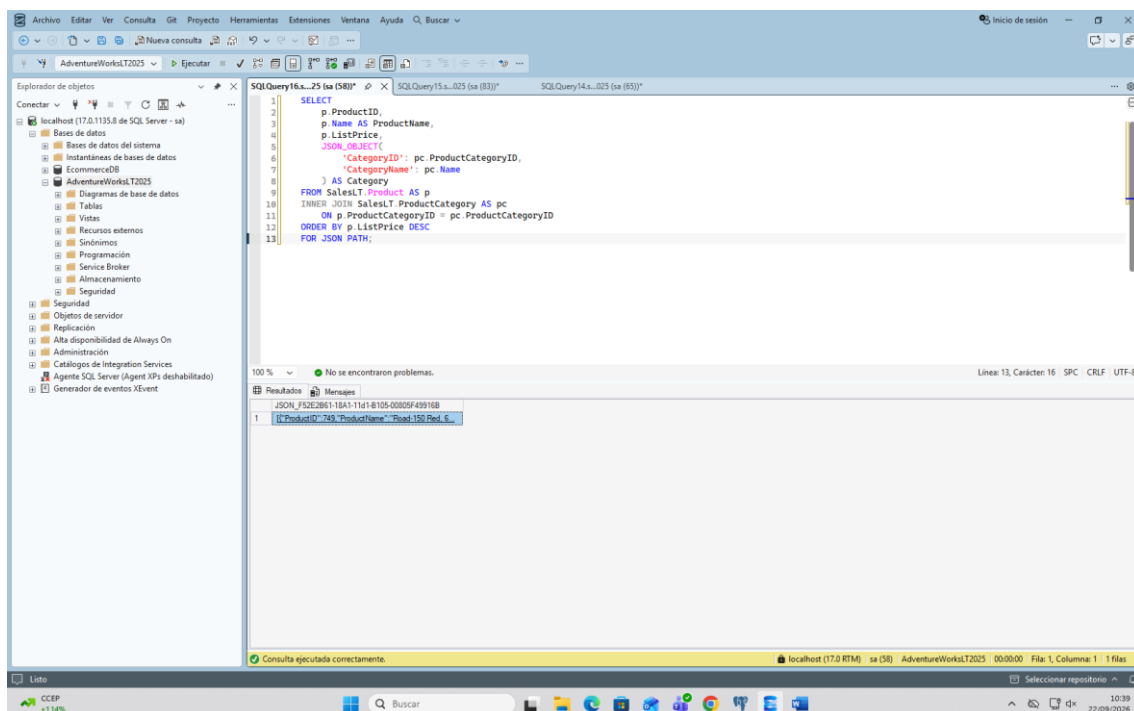
Cada consulta debería devolver hasta cinco filas de datos de ejemplo. Si alguna consulta no devuelve filas o falla, comprueba que la base de datos *AdventureWorksLT* se haya restaurado correctamente y que tengas permisos de lectura.

Crear un objeto JSON para cada producto



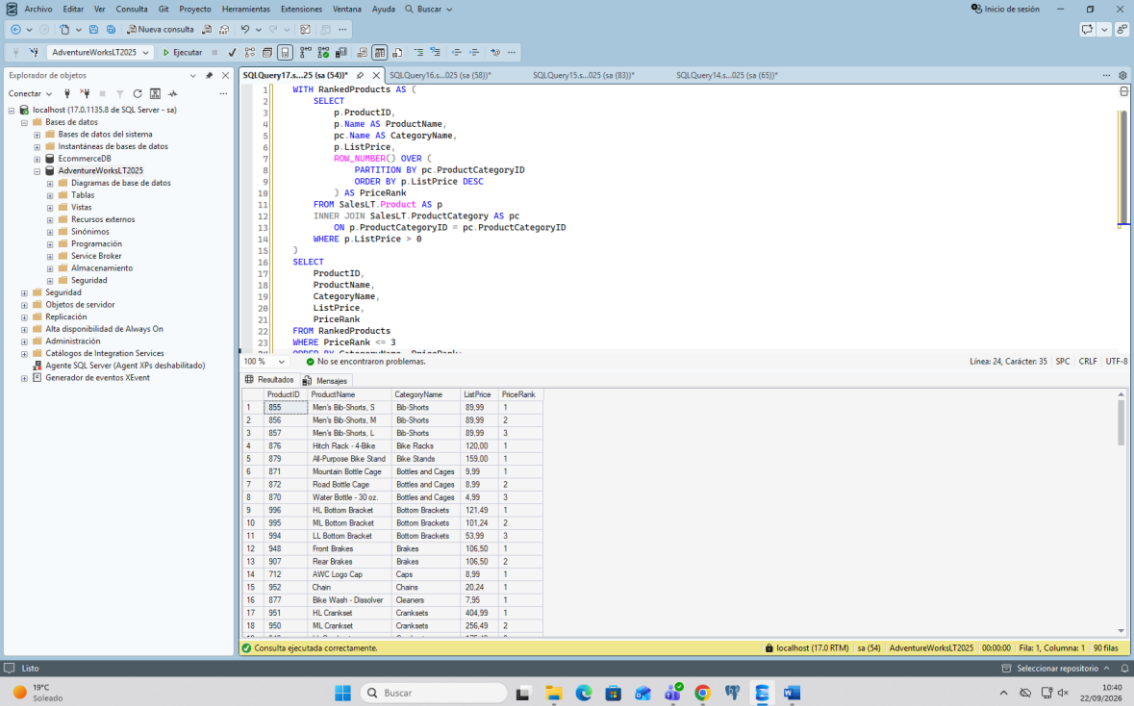
Esta consulta selecciona los productos que tienen un valor de color y formatea los resultados como un array JSON. Cada fila se convierte en un objeto JSON con propiedades que coinciden con los nombres de las columnas. La cláusula FOR JSON PATH se encarga de realizar la conversión automáticamente.

Crear JSON anidado con categorías de productos



Esta consulta utiliza JSON_OBJECT para construir una estructura anidada. La propiedad Category contiene su propio objeto JSON con CategoryID y CategoryName. Este enfoque mantiene los datos relacionados agrupados en el resultado.

Escribir una CTE con una función de ventana para clasificar resultados



The screenshot shows the SQL Server Enterprise Manager interface. The query window displays the following SQL code:

```
1 WITH RankedProducts AS (
2     SELECT
3         p.ProductID,
4         p.Name AS ProductName,
5         pc.Name AS CategoryName,
6         p.ListPrice,
7         ROW_NUMBER() OVER (
8             PARTITION BY pc.ProductCategoryID
9             ORDER BY p.ListPrice DESC
10        ) AS PriceRank
11    FROM SalesLT.Product AS p
12    INNER JOIN SalesLT.ProductCategory AS pc
13        ON p.ProductCategoryID = pc.ProductCategoryID
14    WHERE p.ListPrice > 0
15 )
16 SELECT
17     ProductID,
18     ProductName,
19     CategoryName,
20     ListPrice,
21     PriceRank
22 FROM RankedProducts
23 WHERE PriceRank <= 3
```

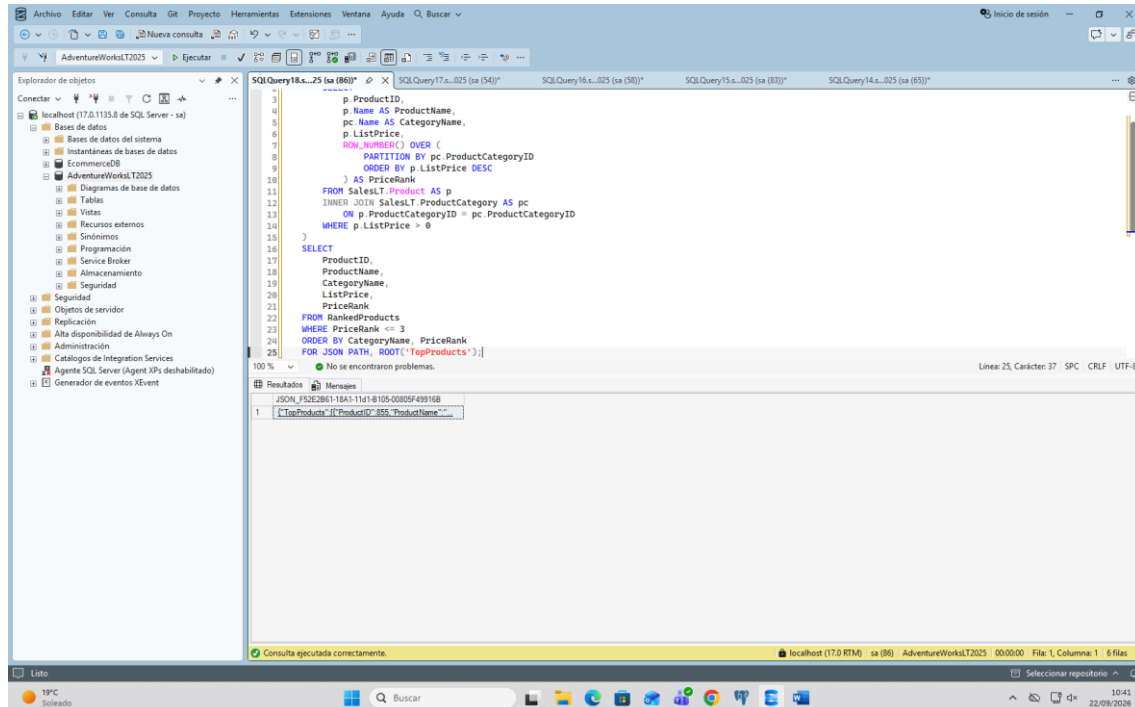
The results grid shows the following data:

ProductID	ProductName	CategoryName	ListPrice	PriceRank
855	Men's Bib-Shorts, S	Bib-Shorts	89.99	1
856	Men's Bib-Shorts, M	Bib-Shorts	89.99	2
857	Men's Bib-Shorts, L	Bib-Shorts	89.99	3
876	Hitch Rack - 4-Bike	Bike Racks	120.00	1
879	AI-Purpose Bike Stand	Bike Stands	159.00	1
871	Mountain Bottle Cage	Bottles and Cages	9.99	1
872	Road Bottle Cage	Bottles and Cages	8.99	2
870	Water Bottle - 30 oz.	Bottles and Cages	4.99	3
996	HL Bottom Bracket	Bottom Brackets	121.49	1
995	HL Bottom Bracket	Bottom Brackets	101.24	2
994	LL Bottom Bracket	Bottom Brackets	53.99	3
940	Front Brakes	Brakes	106.50	1
907	Rear Brakes	Brakes	106.50	2
712	AWC Logo Cap	Caps	8.99	1
952	Chain	Chains	20.24	1
877	Bike Wash - Dissolver	Cleaners	7.95	1
991	HL Crankset	Cranksets	404.99	1
990	HL Crankset	Cranksets	296.49	2

La CTE calcula una clasificación de precios para cada producto dentro de su categoría. La cláusula PARTITION BY reinicia la numeración para cada categoría, y ORDER BY ListPrice DESC asigna el rango 1 al producto más caro. La consulta externa filtra los resultados para mostrar únicamente los 3 productos con mayor precio de cada categoría.

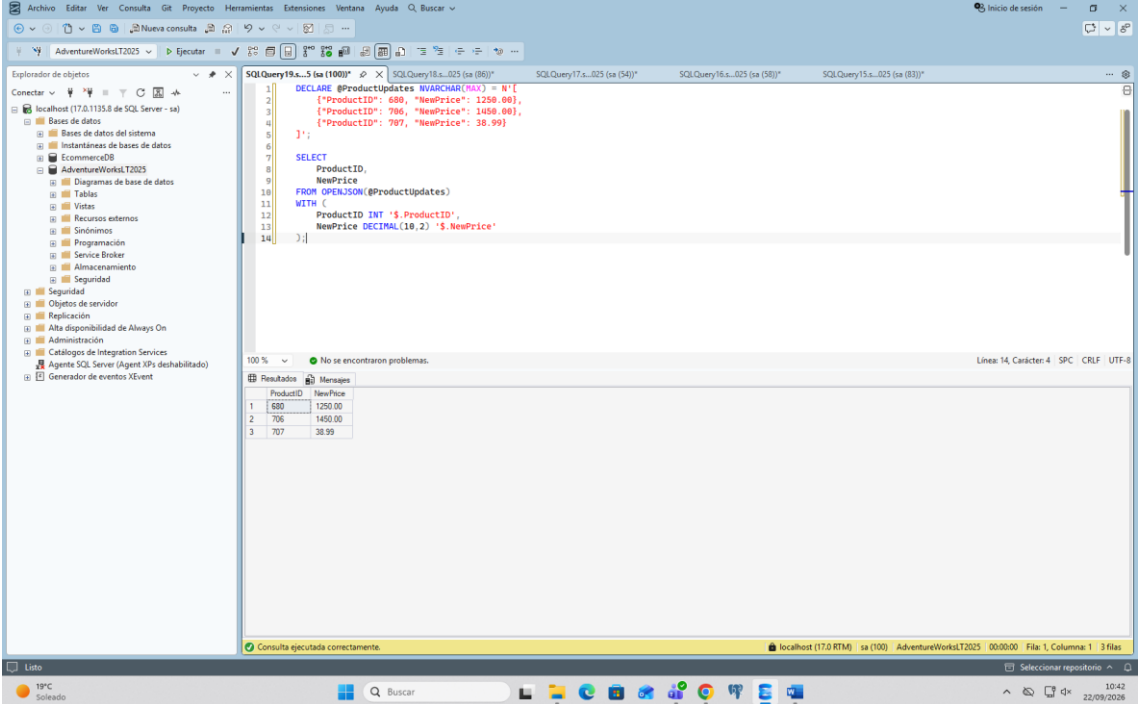
Escribir una CTE con una función de ventana para clasificar resultados

Mostrar los productos clasificados en formato JSON



Añadir ROOT('TopProducts') envuelve todo el array JSON en un objeto con una propiedad con nombre. Esto facilita trabajar con el resultado en aplicaciones que esperan un elemento raíz.

Analizar un array JSON y convertirlo en filas



```
1 DECLARE @ProductUpdates NVARCHAR(MAX) = N'
2 [{"ProductID": 680, "NewPrice": 1250.00},
3 [{"ProductID": 706, "NewPrice": 1450.00},
4 [{"ProductID": 707, "NewPrice": 38.99}
5 ]';
6
7
8 SELECT
9     ProductID,
10     NewPrice
11 FROM OPENJSON(@ProductUpdates)
12 WITH (
13     ProductID INT '$.ProductID',
14     NewPrice DECIMAL(10,2) '$.NewPrice'
15 );
```

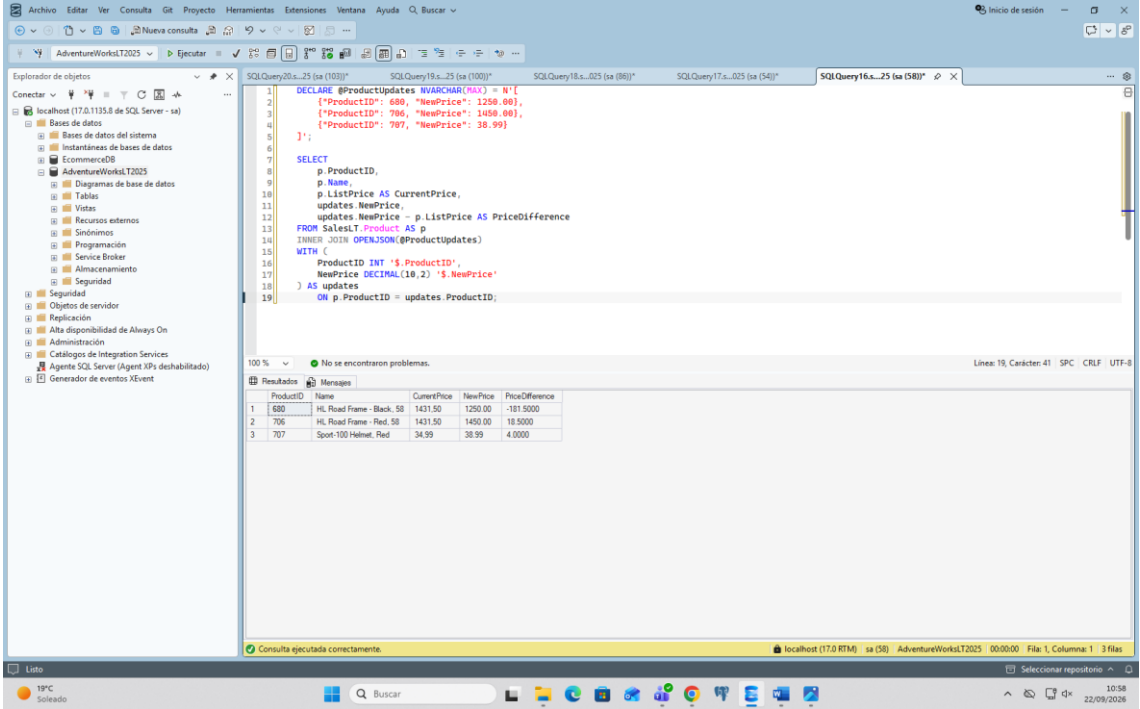
100 % No se encontraron problemas. Línea: 14, Carácter: 4 SPC CRLF UTF-8

	ProductID	NewPrice
1	680	1250.00
2	706	1450.00
3	707	38.99

Consulta ejecutada correctamente. localhost (17.0 RTM) sa (100) AdventureWorksLT2025 00:00:00 File: 1, Columna: 1 3 filas

La cláusula `WITH` define el esquema de salida. Cada propiedad JSON se asigna a una columna con un tipo de datos específico. La sintaxis `$.PropertyName` indica a SQL Server qué ruta JSON debe leer para cada columna.

Combinar el JSON analizado con los datos existentes



```
1 DECLARE @ProductUpdates NVARCHAR(MAX) = N'
2 [{"ProductID": 680, "NewPrice": 1250.00},
3 [{"ProductID": 706, "NewPrice": 1450.00},
4 [{"ProductID": 707, "NewPrice": 38.99}
5 ]';
6
7
8 SELECT
9 p.ProductID,
10 p.Name,
11 p.ListPrice AS CurrentPrice,
12 updates.NewPrice,
13 updates.NewPrice - p.ListPrice AS PriceDifference
14 FROM SalesLT.Product AS p
15 INNER JOIN OPENJSON(@ProductUpdates)
16 WITH (
17 ProductID INT '$.ProductID',
18 NewPrice DECIMAL(10,2) '$.NewPrice'
19 ) AS updates
20 ON p.ProductID = updates.ProductID;
```

ProductID	Name	CurrentPrice	NewPrice	PriceDifference	
1	680	HL Road Frame - Black, 58	1431.50	1250.00	-181.5000
2	706	HL Road Frame - Red, 58	1431.50	1450.00	18.5000
3	707	Spot-100 Helmet, Red	34.99	38.99	4.0000

This query joins the parsed p JSON directly with the Product table. The OPENJSON function with a WITH clause acts like a table, so you can join it just like any other data source. The result shows each product's current price alongside the proposed new price.